

CSC4043

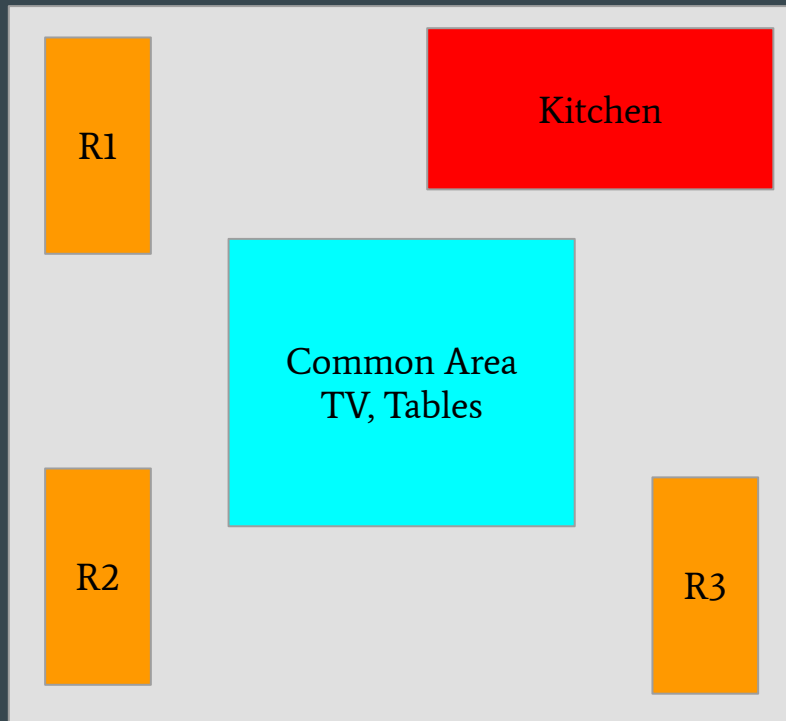
...

Parallel Processing Lab

Content

- In high performance computing , there are ways to handle parallel work
- OpenMP (Shared Memory) - All see the same memory.
- CUDA GPU
- MPI (Distributed memory)

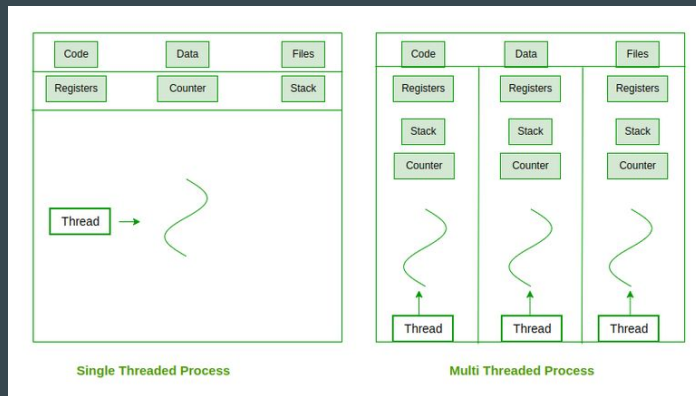
Threads



- Threads are people - Each can do their own work independently.
- Shared Access - Everyone can access these (TV, Kitchen)
- Parallel Access - Many can watch the same TV.
- Communication - People can leave notes in the kitchen for others. Threads communicate by writing to shared variables in memory.

Threads

- A thread is a small unit of work that runs independently inside a program.
- Multiple threads can run at the same time to make a program faster.
 - One thread => One worker doing all tasks alone
 - Multiple threads => many workers sharing the tasks together.

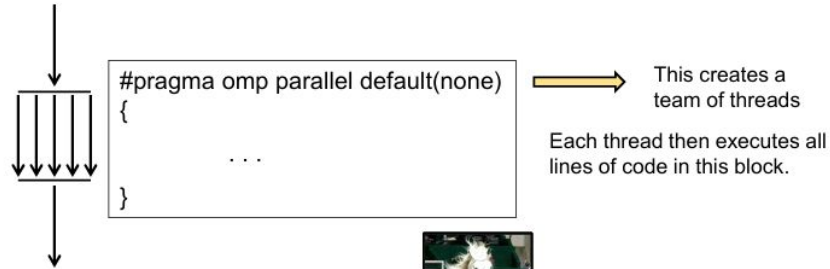


Source: [geeksforgeeks.org](https://www.geeksforgeeks.org)

Parallel Programming using OpenMP

- OpenMP stands for “Open Multi Processing”
- OpenMP is a multi-vendor standard to perform shared-memory multithreading.
- It is both compiler directive and library based
- OpenMP threads share a single executable, a single global memory and a single heap (malloc, new)
- But each OpenMP thread has its own stack (function arguments, function return address, local variables)

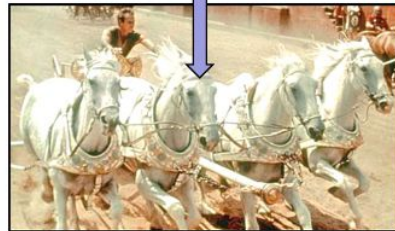
Creating an OpenMP Team of threads



Think of it this way:

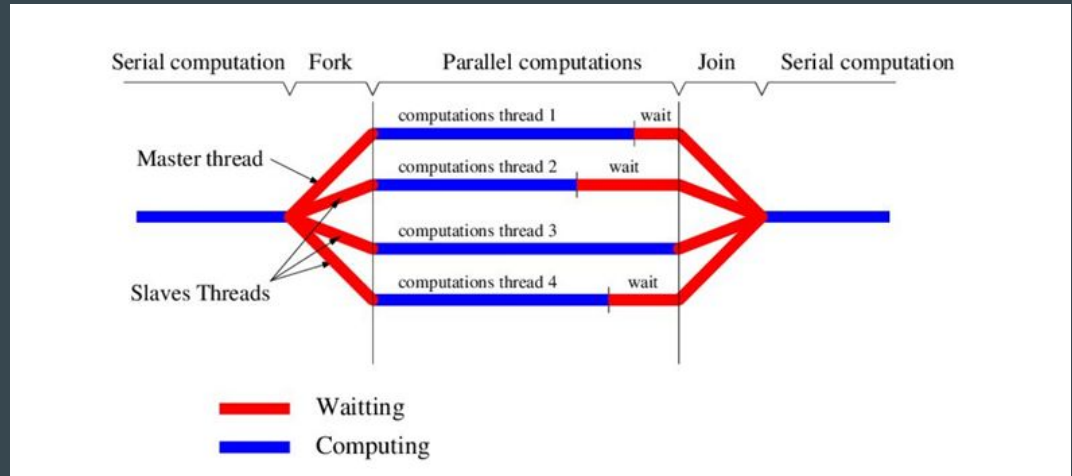


#pragma omp parallel default(none)



Fork-Join model in OpenMP

- Program begins with a master thread.
- At the parallel region, the master thread spawns/ forks other threads.
- At the end of the parallel region, the other threads stop (join/ merge into the master thread)



Hello World with Threads

```
#include <omp.h>      <= library
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{
    #pragma omp parallel    <= compiler directive, tells computer to create team of
                           Threads to run the code block.
    {
        printf("Hello World... from thread = %d\n", omp_get_thread_num());
    }
}
```


How to compile and run?

- Compile

```
gcc -fopenmp filename.c -o filename.out
```

- Run

```
./filename.out
```

Output : Run the program multiple times....

Output 1

Hello World... from thread = 3
Hello World... from thread = 7
Hello World... from thread = 5
Hello World... from thread = 11
Hello World... from thread = 14
Hello World... from thread = 8
Hello World... from thread = 12
Hello World... from thread = 19
Hello World... from thread = 18
Hello World... from thread = 13
Hello World... from thread = 0
Hello World... from thread = 16
Hello World... from thread = 17
Hello World... from thread = 15
Hello World... from thread = 1
Hello World... from thread = 2
Hello World... from thread = 4
Hello World... from thread = 9
Hello World... from thread = 10
Hello World... from thread = 6

Output 2

Hello World... from thread = 4
Hello World... from thread = 16
Hello World... from thread = 9
Hello World... from thread = 19
Hello World... from thread = 1
Hello World... from thread = 2
Hello World... from thread = 17
Hello World... from thread = 0
Hello World... from thread = 3
Hello World... from thread = 15
Hello World... from thread = 14
Hello World... from thread = 5
Hello World... from thread = 6
Hello World... from thread = 13
Hello World... from thread = 8
Hello World... from thread = 18
Hello World... from thread = 12
Hello World... from thread = 10
Hello World... from thread = 7
Hello World... from thread = 11

Why this output?

- `#pragma omp parallel` creates a team of threads. Each thread executes the code block independently.
- `omp_get_thread_num()` returns the unique ID of the calling thread (0, 1, 2....)
- Threads are scheduled by the OS and there is no guarantee which one runs first.
- The number of threads defaults to the number of CPU cores on your machine.

Note: When running the program several times you will observe that the order changes every time. This non determinism is fundamental to parallel programming.

Getting the Thread Count

- PART A - Declaring the variables inside the parallel block

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{
    #pragma omp parallel
    {
        int numt = omp_get_num_threads();    // Get the total number of OMP threads

        int tid = omp_get_thread_num();      // Get the thread id of this thread
        // print "Hello world" along with thread id and number of threads
        printf("Hello World from thread %d of %d.\n", tid, numt);
    }
    return 0;
}
```

Getting the Thread Count

- PART B - Declaring the variables outside the parallel block

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{
    int numt, tid;    // declaring numt and tid outside parallel block
    #pragma omp parallel
    {
        numt = omp_get_num_threads();    // Get the total number of OMP threads

        tid = omp_get_thread_num();      // Get the thread id of this thread

        printf("Hello World from thread %d of %d.\n", tid, numt);
    }

    return 0;
}
```

Why the Difference?

- Part B introduces a RACE condition on the tid variable, Multiple threads write to the same memory location. The output may look correct but we cannot rely on it. Par A is a safe option since it has its own private copy of numt and tid.
- Summary
 - Part A
 - Variables declared inside a parallel block are automatically PRIVATE - each thread gets its own independent copy.
 - Part B
 - Variables declared outside are SHARED by default. All threads read and write to the same memory.
- Both may print correctly here because tid is written and immediately read by the same thread, but it is undefined behaviour.

TRY => Add a loop inside the parallel block that introduces a delay : You will see garbled output in Part B

TRY =>
Add a loop
inside the
parallel
block that
introduces
a delay :
You will see
garbled
output in
Part B

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{
    // declaring numt and tid outside parallel block
    int numt, tid;
    #pragma omp parallel
    {
        int j;
        numt = omp_get_num_threads();
        tid = omp_get_thread_num();
        // introduce wait so that other threads get scheduled
        for(j=0; j<100000000; j++){
            printf("Hello World from thread %d of %d\n", tid, numt);
        }
    }
    return 0;
}
```

Explicit Private and Shared Variables.

Explicitly
control which
variables are
private (per
thread) and
which are shared
(across all
threads)

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{
    int numt, tid;

    #pragma omp parallel private (tid) shared (numt) // specifying private and
shared variables
    {
        numt = omp_get_num_threads();
        tid = omp_get_thread_num();
        printf("Hello World from thread %d of %d.\n", tid, numt);
    }

    return 0;
}
```


Output

Hello World from thread 19 of 20.
Hello World from thread 11 of 20.
Hello World from thread 4 of 20.
Hello World from thread 13 of 20.
Hello World from thread 10 of 20.
Hello World from thread 18 of 20.
Hello World from thread 1 of 20.
Hello World from thread 0 of 20.
Hello World from thread 2 of 20.
Hello World from thread 14 of 20.
Hello World from thread 6 of 20.
Hello World from thread 15 of 20.
Hello World from thread 7 of 20.
Hello World from thread 3 of 20.
Hello World from thread 5 of 20.
Hello World from thread 12 of 20.
Hello World from thread 16 of 20.
Hello World from thread 8 of 20.
Hello World from thread 9 of 20.
Hello World from thread 17 of 20.

- `private(tid)` - each thread gets its own copy of `tid`, initialized to 0. Then thread then sets it via `omp_get_thread_num()`. “No RACE condition”
- `shared(numt)` - all threads share one copy of `numt`. Since all threads write the same value (total thread count) this is safe.
- In general write to a shared variables only when all threads write the same value, or when you use synchronization. (critical sections)
-

Summary

		Use When
Private (x)	Each thread gets its own copy of X (uninitialized)	Thread specific data like IDs, loop counters
shared(x)	All threads access the same x	Read only data or synchronized writes
firstprivate(x)	Private copy, initialized to outer value	When threads need the original value.
default(none)	Forces you to declare every variable explicitly	Best for large programs.

Limiting the Number of Threads

- Use num_threads()

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{

    #pragma omp parallel num_threads(9)
    {
        printf("Hello World... from thread = %d\n", omp_get_thread_num());
    }

    return 0;
}
```

num_threads(N) overrides the OMP_NUM_THREADS environment variable for that block

Three ways to set thread count (Priority Order)

- `num_threads(N)` on pragma
- `omp_set_num_threads(N)` function call before the parallel block
- `OMP_NUM_THREADS=N` ./program environment variable

Parallel For Loop

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[])
{

    #pragma omp parallel for num_threads(9)
    for(int i=0; i<=6; i++){
        printf("Hello World.. from thread = %d\n", omp_get_thread_num());
    }

    return 0;
}
```

Parallel Loops

- The loop runs 7 iterations ($i=0$ to 6).
- We have requested 9 threads, So only 7 threads actually do the work, each handles exactly one iteration.
- In OpenMP it divides the iterations using a scheduling policy, Default is static, iterations splits evenly among threads.

Note

- Only use `#pragma omp parallel` for when each loop iteration is INDEPENDENT. No iteration should depend on the result of another. If there is a dependency, you will get incorrect results.

Timing

```
#include <stdio.h>
#include <omp.h>

int main(){
    int size = 1000000;
    int A[size];

    double start_time = omp_get_wtime();

    #pragma omp parallel for
    for(int i=0; i<size;i++){
        A[i] = i * i;
    }

    double end_time = omp_get_wtime();
    printf("Time taken = %f seconds\n", end_time - start_time);
}
```

Run the program multiple times.
You will observe small variations in timing
due to OS scheduling.

Parallel Sections

- Assign different independent code blocks to different threads using sections.

```
int main(int argc, char* argv[])
{
    #pragma omp parallel sections num_threads(8)
    {
        #pragma omp section
        {
            printf("Hello World.. from thread = %d\n", omp_get_thread_num());
        }

        #pragma omp section
        {
            printf("Hello World.. from thread = %d\n", omp_get_thread_num());
        }

        #pragma omp section
        {
            printf("Hello World.. from thread = %d\n", omp_get_thread_num());
        }
    }
}
```


Parallel Sections

- There are 3 sections and upto 8 threads. Each section is executed exactly ONE thread.
- Only 3 threads do work (one per section). The other 5 threads are idle.
- Which thread ID handles which section is decided by RUNTIME, where it changes at every run.

Array addition with Sections

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

void main()
{
    int a[8] = {1, 5, 8, 10, 3, 7, 2, 1};
    int b[8] = {2, 3, 4, 9, 0, 3, 2, -1};
    int c[8];

    int tid;

    #pragma omp parallel sections num_threads(8)
    {
        tid = omp_get_thread_num();
        c[tid] = a[tid] + b[tid];
        printf("c[%d] = %d\n",tid, c[tid]);
    }
}
```

Expected outputs

C[0] = 3

C[1] = 8

C[2] = 12

C[3] = 19

C[4] = 3

C[5] = 10

C[6] = 4

C[7] = 0

Array addition with Sections

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

void main()
{
    int a[8] = {1, 5, 8, 10, 3, 7, 2, 1};
    int b[8] = {2, 3, 4, 9, 0, 3, 2, -1};
    int c[8];

    int tid;

    #pragma omp parallel sections num_threads(8)
    {
        tid = omp_get_thread_num();
        c[tid] = a[tid] + b[tid];
        printf("c[%d] = %d\n",tid, c[tid]);
    }
}
```

To compute all 8 elements you can use omp parallel as well,

```
#pragma omp parallel for num_threads(8)
for(int i=0; i<8; i++){
    C[i] = a[i] + b[i];
    printf("c[%d] = %d\n", i, c[i]);
}
```

Array addition with Sections

Use `SECTIONS` when you have a small, fixed number of different tasks to run in parallel.

Use `parallel` for when you are distributing iterations of the `SAME` loop.

Reduction (Safe Parallel Summation)

Race condition in parallel accumulation and how to fix them with reduction.

Version 1 - Sequential (Slow)

```
#include <stdio.h>
#include <omp.h>

int main(){
    double start_time = omp_get_wtime();

    double sum = 0.0;
    for(int i=1; i<=1000; i++){
        sum += i;
    }

    double end_time = omp_get_wtime();
    printf("Time taken = %f seconds\n", end_time - start_time);
    printf("Sum = %f\n", sum);
}
```

Reduction (Safe Parallel Summation)

Race condition in parallel accumulation and how to fix them with reduction.

Version 2 - Parallel
without
reduction(WRONG)

```
#include <stdio.h>
#include <omp.h>

int main(){
    double start_time = omp_get_wtime();

    double sum = 0.0;

    #pragma omp parallel for
    for(int i=1; i<=1000; i++){
        sum += i;
    }

    double end_time = omp_get_wtime();
    printf("Time taken = %f seconds\n", end_time - start_time);
    printf("Sum = %f\n", sum);
}
```

Reduction (Safe Parallel Summation)

Race condition in parallel accumulation and how to fix them with reduction.

Version 3 - Parallel
with REDUCTION

```
#include <stdio.h>
#include <omp.h>

int main(){
    double start_time = omp_get_wtime();

    double sum = 0.0;

    #pragma omp parallel for reduction(+:sum)
    for(int i=1; i<=1000; i++){
        sum += i;
    }

    double end_time = omp_get_wtime();
    printf("Time taken = %f seconds\n", end_time - start_time);
    printf("Sum = %f\n", sum);
}
```

How Reduction Works

- Each thread gets its own PRIVATE copy of sum, initialized to 0
- Each thread computes a partial sum over its assigned iterations.
- At the end of the parallel region, all private copies are combined (reduced) using + into the original sum variable.
- No RACE CONDITION - threads never share their local copy during computation.

What OpenMP isn't

- Does not check for data dependencies, data conflicts, deadlocks, race conditions. You are responsible for avoiding those yourself.
- Does not guarantee identical behaviour across vendors or hardware, or even between multiple runs on the same vendor's hardware.
- Does not guarantee the order in which threads execute, just that they do execute.
- Is not overhead free.

SUMMARY

Concept / Pragma	What It Does
<code>#pragma omp parallel</code>	Creates a team of threads; all execute the block
<code>num_threads(N)</code>	Limits the team size to N threads
<code>#pragma omp parallel for</code>	Splits loop iterations across the thread team
<code>#pragma omp parallel sections</code>	Assigns distinct code blocks to different threads
<code>private(var)</code>	Each thread gets its own uninitialized copy
<code>shared(var)</code>	All threads access the same variable (be careful!)
<code>firstprivate(var)</code>	Private copy initialized to the original value
<code>reduction(op:var)</code>	Safe parallel accumulation — prevents race conditions
<code>omp_get_thread_num()</code>	Returns the current thread's ID (0 to N-1)
<code>omp_get_num_threads()</code>	Returns total number of threads in the team
<code>omp_get_wtime()</code>	Returns current wall-clock time in seconds

How to specify how many OpenMP threads you want to have available?

```
omp_set_num_threads(num);
```

Asking how many cores this program has access to:

```
Num = omp_get_num_procs();
```

Setting the number of available threads to the exact number of cores available:

```
omp_set_num_threads (omp_get_num_proc());
```

Asking how many OpenMP threads this program is using right now:

```
num = omp_get_num_threads();
```

Asking which thread number this one is

```
me = omp_get_thread_num();
```

END OF THE SESSION 01